

How would you define Machine Learning?

General Definition: At its core, Machine Learning is the science and art of programming computers so they can learn from data.

Supervised and Unsupervised Learning

1. Supervised Learning

Supervised learning is a type of **Machine Learning** where the training dataset contains both **input data** and **correct outputs (labels)**. The model learns by using these labeled examples to predict outcomes for new, unseen data.

In supervised learning, every training sample has an expected answer, helping the model understand the relationship between inputs and outputs.

Common Tasks in Supervised Learning

1. Classification

Classification is used to assign data into predefined categories or classes.

Example:

A spam detection system learns from emails labeled as **spam** or **not spam (ham)** and predicts whether a new email belongs to either category.

Applications:

- Email spam detection
- Disease diagnosis
- Image classification
- Sentiment analysis

2. Regression

Regression is used to predict a **continuous numerical value**.

Example:

Predicting the price of a car using features such as:

- Mileage

- Age
- Brand
- Engine condition

Applications:

- House price prediction
- Stock market forecasting
- Sales prediction
- Weather forecasting

Popular Supervised Learning Algorithms

- **Linear Regression** – predicts continuous numerical values
- **Logistic Regression** – used for classification problems
- **k-Nearest Neighbors (KNN)** – classifies data based on nearby examples
- **Support Vector Machines (SVMs)** – finds optimal boundaries between classes
- **Decision Trees** – decision-based prediction model
- **Random Forests** – ensemble of multiple decision trees for improved accuracy
- **Neural Networks** – powerful models for complex learning tasks such as image and speech recognition

2. Unsupervised Learning

Unsupervised learning is a type of **Machine Learning** where the training data is **unlabeled**. Unlike supervised learning, there are no predefined outputs or “correct answers.”

The system automatically discovers hidden patterns, relationships, and structures within the data.

Key Techniques in Unsupervised Learning

1. Clustering

Clustering groups similar data points into clusters based on shared characteristics.

K-Means Clustering

K-Means is a simple and efficient clustering algorithm that:

- Divides data into a predefined number of clusters (**K**)
- Finds the center (**centroid**) of each cluster

- Assigns data points to the nearest centroid

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN:

- Groups data based on density
- Detects clusters of any shape
- Handles outliers effectively

Applications of Clustering:

- Customer segmentation
 - Market analysis
 - Image segmentation
 - Recommendation systems
-

2. Anomaly and Novelty Detection

These methods identify unusual or abnormal data points.

Anomaly Detection

The model learns what “normal” data looks like and flags unusual observations as anomalies.

Examples:

- Fraud detection in banking
- Network intrusion detection
- Equipment failure monitoring

Novelty Detection

Novelty detection assumes the training data is clean and focuses on identifying entirely new or unseen patterns.

Common Algorithms

- **One-Class SVM**
 - **Isolation Forest**
 - **Gaussian Mixture Models (GMMs)**
-

3. Visualization and Dimensionality Reduction

These techniques simplify high-dimensional datasets while preserving important information.

Visualization

High-dimensional data is converted into **2D or 3D representations** to help humans understand patterns and relationships.

Dimensionality Reduction

Reduces the number of features to:

- Improve model efficiency
- Reduce training time
- Handle the **curse of dimensionality**

Popular Techniques

Principal Component Analysis (PCA)

PCA reduces dimensions by finding directions that preserve maximum data variance.

Locally Linear Embedding (LLE)

LLE is a manifold learning technique that helps unfold complex or twisted data structures into simpler forms.

4. Autoencoders

Autoencoders are neural networks that learn compressed representations (**latent representations**) of input data.

They contain:

- **Encoder** → Compresses data into a lower-dimensional representation
- **Decoder** → Reconstructs the original input from compressed data

Applications:

- Dimensionality reduction
 - Feature extraction
 - Data compression
 - Data generation
-

5. Association Rule Learning

Association rule learning identifies hidden relationships between variables in large datasets.

Example:

A supermarket may discover that customers who buy **barbecue sauce** also frequently buy **steak**, helping improve product placement strategies.

Applications:

- Market basket analysis
 - Product recommendations
 - Consumer behavior analysis
-

6. Density Estimation

Density estimation determines the probability distribution of a dataset.

It helps:

- Understand data distribution
- Detect anomalies
- Identify low-density regions containing unusual instances

Common Technique

Gaussian Mixture Models (GMMs) are commonly used for density estimation.

=====

Can you name four types of problems where it shines?

Machine Learning is particularly effective for several types of challenges where traditional programming methods often struggle. According to the sources, here are four types of problems where it shines:

- **Problems requiring long lists of rules:** For tasks where a traditional approach would result in a long and complex set of hand-tuned rules, a Machine Learning algorithm can often simplify the code and provide better performance. A classic example is a spam filter, which is much easier to maintain when built with ML rather than manually written rules

- **Complex problems with no traditional solution:** For problems that are too complex for traditional approaches or have no known effective algorithm—such as speech recognition—the best Machine Learning techniques can often find a solution by learning from examples
- **Fluctuating environments:** In environments where data is constantly changing, Machine Learning systems can automatically adapt to new data without requiring constant manual intervention. For instance, a spam filter can automatically learn to identify new patterns used by spammers as they attempt to bypass existing filters
- **Gaining insights from large datasets:** Applying Machine Learning techniques to analyze massive amounts of data can help humans discover patterns and trends that were not immediately apparent. This process, often referred to as data mining, leads to a better understanding of complex problems.

=====

What is a labeled training set?

A labeled training set is a dataset used to train a Machine Learning model where each training instance (or sample) includes the desired solution, known as a label

In this type of training set, the system is provided with many examples that already have the correct answers attached, allowing the algorithm to learn the patterns necessary to make predictions on new, unseen data

Key characteristics include:

Supervised Learning: Labeled training sets are the core requirement for supervised learning systems.

Target Values: In classification tasks, the label is the category or class (e.g., "spam" or "ham" for an email)

In regression tasks, the label is a target numeric value (e.g., the price of a car or a house)

Training Instances: Every individual example within the set consists of its features (the data used for prediction) and its corresponding label (the correct output).

=====

What are the two most common supervised tasks?

The two most common supervised learning tasks are regression and classification

Regression: The goal of this task is to predict a target numeric value, such as the price of a car or a house, based on a set of input features (also known as predictors)

Classification: In this task, the system is trained to predict classes

A classic example is a spam filter, which is trained using many example emails along with their labels ("spam" or "ham") and must learn how to correctly categorize new incoming messages

While these are the two primary tasks, some algorithms can be used for both; for example, Logistic Regression is a regression algorithm commonly used for classification because it can output a value representing the probability that an instance belongs to a specific class

=====

Can you name four common unsupervised tasks?

According to the sources, four common unsupervised learning tasks include:

Clustering: The goal is to identify similar instances and group them together into clusters

It is used for applications such as customer segmentation, data analysis, and image segmentation

Anomaly Detection (and Novelty Detection): This task involves learning what "normal" data looks like and then identifying instances that deviate significantly from the norm

It is commonly used for fraud detection or identifying defective products in manufacturing

Visualization and Dimensionality Reduction: These tasks aim to simplify complex, high-dimensional data without losing too much information

Visualization algorithms often output a 2D or 3D representation that can be plotted to help humans understand how data is organized, while dimensionality reduction can speed up training for other machine learning algorithms

Association Rule Learning: The objective is to dig into large amounts of data to discover interesting relations between attributes

For example, a supermarket might use this to find that customers who buy barbecue sauce often also buy steak, allowing them to optimize product placement

Another important unsupervised task mentioned in the sources is density estimation, which is the task of estimating the probability density function of the process that generated a dataset; it is often used as a tool for anomaly detection

=====

What type of Machine Learning algorithm would you use to allow a robot to walk in various unknown terrains?

Reinforcement Learning is the most appropriate type of Machine Learning algorithm for teaching a robot to walk in various unknown terrains

This approach is best suited for this task for several reasons:

- Trial and Error: In Reinforcement Learning, an agent (the robot) learns by interacting with its environment. It observes the state of the terrain, selects and performs actions (such as moving a motor), and receives rewards or penalties based on the outcome
 - Policy Development: Through this process, the robot learns a policy—a strategy that defines the best action to take in a given situation to maximize its rewards over time
 - For a walking robot, rewards might be granted for making forward progress, while penalties (negative rewards) would be given for falling over or wasting time
 - Natural Fit for Complex Movement: While it might be possible to express walking as a supervised or semisupervised problem, the sources note that Reinforcement Learning is a much more natural and effective fit for this specific type of challenge
 - In practice, training for such tasks often begins in a simulated environment (like OpenAI Gym or PyBullet) because training in the real world can be slow, expensive, and dangerous—for instance, a robot could be damaged if it falls off a cliff during the learning process
 - Once the agent has "bootstrapped" its learning in simulation, the resulting policy can be applied to the physical robot.
- =====

What type of algorithm would you use to segment your customers into multiple groups?

To segment customers into multiple groups, the most appropriate choice is a clustering algorithm, which is a type of unsupervised learning

The selection of a specific approach depends on whether you have already defined the groups you want:

- When groups are not predefined: You should use clustering. The algorithm will analyze your data (such as customer purchases or website activity) to identify connections and group similar individuals together on its own. Common techniques for this include K-Means, DBSCAN, and Gaussian mixture models
- When groups are already known: If you have a set of established categories and a labeled training set (examples of customers who belong to each category), you can use a classification algorithm (supervised learning) to assign new customers into these existing groups
- Clustering is particularly valuable for customer segmentation because it allows you to discover natural patterns in your data, which can then be used to design targeted marketing strategies for each specific segment

Would you frame the problem of spam detection as a supervised learning problem or an unsupervised learning problem?

Spam detection is framed as a supervised learning problem. This classification is based on the following characteristics:

- **Labeled Training Data:** The algorithm is trained using a dataset where each email is already marked with its "desired solution," known as a label (e.g., "spam" or "ham")
- **Classification Task:** It is a typical classification task where the system learns from these labeled examples to recognize patterns—such as unusually frequent words or phrases—that distinguish spam from legitimate mail
- **Prediction:** Once trained, the model uses what it has learned to assign new, unseen emails to one of the two predefined categories

The sources note that while a modern spam filter may also be characterized as an online or model-based system, its fundamental reliance on human-provided supervision (labels) makes it a supervised learning system

What is an online learning system?

An online learning system is a Machine Learning system that can learn incrementally by being fed data instances sequentially, either individually or in small groups called mini-batches. This contrasts with batch learning, where a system must be trained using all available data at once

Key features and benefits of online learning include:

- **Adaptability:** It is ideal for systems that receive a continuous flow of data, such as stock prices, and need to adapt to changes rapidly or autonomously
- **Resource Efficiency:** Once the system has learned from new data instances, it no longer needs them and they can be discarded (unless rollback capabilities are required). This significantly reduces the amount of storage space needed
- **Handling Large Datasets:** It can be used for out-of-core learning, which involves training on huge datasets that are too large to fit into a single machine's main memory. The algorithm loads a portion of the data, runs a training step, and repeats this until it has processed the entire set
- **Learning Rate:** A critical parameter for these systems is the learning rate, which determines how quickly the system should adapt to changing data. A high learning rate allows the system to adapt fast but may cause it to forget old data quickly, while a low learning rate makes the system more stable but less sensitive to new patterns or noise. A significant challenge of online learning is that bad data (such as from a malfunctioning sensor or malicious spamming) can cause the system's performance to decline gradually.

To mitigate this risk, these systems require close monitoring to detect performance drops and potentially trigger a "switch off" of the learning process or a reversion to a previous state.

=====

What is out-of-core learning?

- Out-of-core learning is a technique used to train Machine Learning systems on vast datasets that are too large to fit into a single machine's main memory (RAM). According to the sources, here is how it works and why it is used:
- **Mechanism:** Instead of loading the entire dataset at once, an out-of-core algorithm chops the data into smaller pieces or mini-batches. It loads one piece at a time, runs a training step, and repeats this process until it has processed the entire dataset.
- **Relationship to Online Learning:** Out-of-core learning utilizes online learning techniques because it learns incrementally. However, the sources note that "online

learning" can be a confusing name in this context because out-of-core training is usually performed offline (not on a live system).

- Examples of Algorithms: Several algorithms discussed in the sources can be implemented for out-of-core learning:
- Stochastic Gradient Descent (SGD): Because it processes only one instance (or a small batch) at a time, it is naturally suited for huge datasets.
- Mini-batch K-Means: A variant of the K-Means clustering algorithm that uses mini-batches to handle datasets that don't fit in memory.
- Incremental PCA: A version of Principal Component Analysis used for large training sets.
- Implementation Tools: To handle these large arrays, programmers often use tools like NumPy's memmap class, which allows them to manipulate data stored in a binary file on disk as if it were in memory, loading only the necessary parts on demand.

=====

What type of learning algorithm relies on a similarity measure to make predictions?

Instance-based learning is the type of algorithm that relies on a similarity measure to make predictions.

In this approach, the system "learns the examples by heart" rather than building a predictive model. When presented with a new, unseen instance, the algorithm uses a similarity measure to compare it to the instances it has already learned (the training set) and finds the most similar ones to determine the output. A primary example of this is the k-Nearest Neighbors algorithm. For instance, in a classification task, a new data point would be assigned to the category that represents the majority of its most similar neighbors. A very basic similarity measure used in instance-based systems, such as a simple spam filter, might involve counting the number of words two emails have in common to determine how alike they are.

=====

What is the difference between a model parameter and a learning algorithm's hyperparameter?

The difference between a model parameter and a learning algorithm's hyperparameter lies in what they belong to and how they are determined during the Machine Learning process.

Model Parameters

Definition: These are internal variables that belong to the model itself and determine how it makes predictions given a new instance.

Determination: They are not set by the user; instead, the learning algorithm finds the optimal values for these parameters during the training process so that the model best fits the training data.

Hyperparameters

Definition: A hyperparameter is a parameter of the learning algorithm, not of the model itself.

Determination: They must be set prior to training and remain constant throughout the training process. The learning algorithm does not change them; rather, they are used to guide or constrain the learning process.

Purpose: They are often used to control the complexity of a model to avoid overfitting (regularization) or to determine the speed of learning.

Examples:

Regularization weight: A parameter (like α) that controls how much to constrain the model parameters to keep them small.

Learning rate: Determines the size of the steps the algorithm takes toward a minimum during optimization.

Neural Network architecture: The number of hidden layers and the number of neurons per hidden layer.

In summary, you tune hyperparameters to help the learning algorithm find the best model parameters

=====

What do model-based learning algorithms search for? What is the most common strategy they use to succeed? How do they make predictions?

Model-based learning algorithms search for the optimal values for model parameters to ensure the model generalizes well to new, unseen instances. The most common strategy they use to succeed is minimizing a cost function, which measures the performance gap between the system's predictions and the actual training data.

To make predictions, the system feeds the features of a new instance into the model's prediction function (also referred to as a hypothesis), utilizing the specific parameter values that were identified during the training process. For example, in a linear model, this involves computing a weighted sum of the input features plus a bias term.

Can you name four of the main challenges in Machine Learning?

Based on the sources, here are four of the main challenges encountered in Machine Learning:

- **Insufficient Quantity of Training Data:** Unlike humans, most Machine Learning algorithms require a significant amount of data to function properly. For simple problems, you typically need thousands of examples, while complex tasks like image or speech recognition can require millions.
- **Nonrepresentative Training Data:** To generalize well to new cases, the training data must be representative of the instances you want to predict. If the training set is too small, you may encounter sampling noise, and even large samples can be problematic if the sampling method is flawed, leading to sampling bias.
- **Poor-Quality Data:** If training data is full of errors, outliers, and noise, it becomes much harder for the algorithm to detect underlying patterns. Data scientists often spend a significant amount of time cleaning data—for example, by discarding outliers or filling in missing values.
- **Irrelevant Features:** A system can only learn effectively if the training data contains enough relevant features and not too many irrelevant ones. The process of feature engineering, which includes selecting, extracting, and creating new features, is critical to a project's success.

Other major challenges mentioned in the sources include overfitting the training data (where the model performs well on training data but poorly on new data) and underfitting the training data (where the model is too simple to learn the underlying structure of the data)

If your model performs great on the training data but generalizes poorly to new instances, what is happening? Can you name three possible solutions?

When a model performs great on the training data but generalizes poorly to new instances, it is overfitting the training data. Overfitting occurs when a model is too complex relative to the amount and noisiness of the training data. In such cases, the model may end up detecting patterns in the random noise itself rather than identifying the true underlying structure, which prevents it from making accurate predictions on unseen data.

According to the sources, three possible solutions to address overfitting are:

- Simplify the model: You can choose a simpler algorithm with fewer parameters (such as a linear model instead of a high-degree polynomial model), reduce the number of features used in the training data, or constrain the model through a process called regularization.
- Gather more training data: Providing the model with a larger quantity of examples can help it better distinguish between real patterns and random noise.
- Reduce noise in the training data: This involves cleaning up the dataset by fixing data errors or removing outliers, which makes it easier for the algorithm to detect the correct underlying patterns.

What is a test set, and why would you want to use it?

A test set is a portion of your data—typically about 20%—that is set aside and not used during the training process.

You would want to use a test set for the following reasons:

- Estimate Generalization Error: Its primary purpose is to estimate the generalization error (also called the out-of-sample error). This value tells you how well your model will perform on instances it has never encountered before.
- Final Evaluation: It provides a reliable way to evaluate the model's performance before it is launched in production.
- Identify Overfitting: The test set helps detect if your model is overfitting the training data. If the training error is low but the generalization error on the test set is high, the model is likely overfitting.
- Avoid Data Snooping Bias: By setting the test set aside and never looking at it until the final evaluation, you avoid data snooping bias. If you look at the test data while selecting a model or tuning hyperparameters, you might find patterns that

lead you to choose a specific model that works well for those specific instances but does not generalize to truly new data.

In summary, the test set acts as an independent benchmark to ensure your model is ready for real-world use and not just memorizing the training examples.

What is the purpose of a validation set?

A validation set (also known as a development set or "dev set") is a portion of the training data that is held out to evaluate candidate models and select the best one.

Its primary purposes include:

- **Model Selection:** When you are hesitating between different types of models—such as a linear model versus a polynomial model—you can train both on the reduced training set and compare their performance on the validation set to decide which one is better.
- **Hyperparameter Tuning:** A validation set is essential for finding the optimal values for a learning algorithm's hyperparameters (such as the amount of regularization to apply). By training multiple variants of a model with different hyperparameter values, you can use the validation set to identify which specific combination results in the best performance.
- **Avoiding Overfitting the Test Set:** If you were to use the test set to tune your hyperparameters, you would measure the generalization error multiple times on that set and adapt your model until it produces the best results for that specific data. This would result in data snooping bias, meaning the model is unlikely to perform as well on truly new data because it has been "optimized" for the test set itself. The validation set acts as an intermediary, allowing you to fine-tune the model while keeping the test set as a clean, independent benchmark for the final evaluation. In cases where the dataset is small, you might use cross-validation instead of a single, fixed validation set. This technique involves splitting the training data into multiple small subsets (folds) and training the model several times, using a different fold for validation each time to get a more accurate and precise measure of performance.

What is the train-dev set, when do you need it, and how do you use it?

The train-dev set is an additional subset of data held out from the training set, used to diagnose performance issues when there is a mismatch between training and production data,.

What is it?

The train-dev set is a portion of the training data (for example, images downloaded from the web) that is set aside and not used to train the model. While the model is trained on the remainder of the training set, the train-dev set serves as a benchmark to see how well the model generalizes to data that comes from the same distribution as its training material,.

When do you need it?

You need a train-dev set when there is a risk of data mismatch—a situation where your training data is not perfectly representative of the data the model will encounter in production,.

A common example is building a mobile app to identify flower species:

Training Data: You might have millions of high-quality flower photos from the web.

Validation/Test Data: These sets must be representative of the actual app usage, so they would consist of lower-quality photos taken by mobile devices.

Without a train-dev set, if your model performs poorly on the validation set, you cannot tell if the failure is because the model overfit the training data or because the web photos are simply too different from the mobile photos.

How do you use it?

To use this set effectively, you follow a specific evaluation process after training your model on the (reduced) training set:

Evaluate on the train-dev set: If the model performs poorly, then it has overfitted the training set,. You should address this by simplifying the model, regularizing it, or gathering more/cleaner training data.

Evaluate on the validation set: If the model performs well on the train-dev set but poorly on the validation set, the problem is data mismatch,. In this case, you should try to preprocess your training data to make it look more like the validation/production data (e.g., adding noise or blur to web photos) and then retrain.

By using this set, you can pinpoint exactly whether your model needs better generalization (fixing overfitting) or better alignment with real-world data (fixing mismatch)

=====

What can go wrong if you tune hyperparameters using the test set?

If you tune hyperparameters using the test set, you risk overfitting that specific dataset, which leads to several critical issues for your Machine Learning model.

The primary problems include:

Optimistic Generalization Error: When you measure performance multiple times on the test set and adapt your hyperparameters to produce the best result for that specific data, your estimate of the generalization error (the error rate on new cases) will be too optimistic.

Data Snooping Bias: This process results in what is called data snooping bias. Your brain (or your algorithm) detects patterns in the test data that lead you to select a specific model or hyperparameter configuration that works well for those instances but may not exist in the broader population.

Poor Real-World Performance: Because the model has been "optimized" for the test set, it is unlikely to perform as well on truly new data once it is launched into production. For example, you might find a model that yields only 5% error on your test set, but once deployed, it produces 15% errors because the improvements were specific to the test data and did not generalize.

To avoid these pitfalls, the sources recommend using a validation set (or "dev set") held out from the training data to select your model and tune its hyperparameters. The test set should remain untouched as a final, independent benchmark to be used only once, at the very end of your project

